and when necessary, a "what's different about this software increment" description should be published.[9]

Principle 5. ***Buggy software should be fixed first, delivered later.*** Under time pressure, some software organizations deliver low-quality increments with a warning to the customer that bugs "will be fixed in the next release." This is a mistake. There's a saying in the software business: "Customers will forget you delivered a high-quality product a few days late, but they will never forget the problems that a low-quality product caused them. The software reminds them every day."

6.3 SUMMARY

Software engineering practice encompasses principles, concepts, methods, and tools that software engineers apply throughout the software process. Every software engineering project is different. Yet, a set of generic principles applies to the process and to the practice of each framework activity regardless of the project or the product.

A set of core principles helps in the application of a meaningful software process and the execution of effective software engineering methods. At the process level, core principles establish a philosophical foundation that guides a software team as it navigates through the software process. At the practice level, core principles establish a collection of values and rules that serve as a guide as you analyze a problem, design a solution, implement and test the solution, and ultimately deploy the software in the user community.

Communication principles focus on the need to reduce noise and improve bandwidth as the conversation between developer and customer progresses. Both parties must collaborate for the best communication to occur.

Planning principles provide guidelines for constructing the best map for the journey to a completed system or product. The plan may be designed solely for a single software increment, or it may be defined for the entire project. Regardless, it must address what will be done, who will do it, and when the work will be completed.

Modeling principles serve as a foundation for the methods and notation that are used to create representations of the software. Modeling encompasses both analysis and design, describing representations of the software that progressively become more detailed. The intent of the models is to solidify understanding of the work to be done and to provide technical guidance to those who will implement the software.

Construction incorporates a coding and testing cycle in which source code for a component is generated and tested. Coding principles define generic actions that should occur before code is written, while it is being created, and after it has been completed. Although there are many testing principles, only one is dominant: Testing is a process of executing a program with the intent of finding an error.

Deployment occurs as each software increment is presented to the customer and encompasses delivery, support, and feedback. Key principles for delivery consider managing customer expectations and providing the customer with appropriate support

9 During the communication activity, the software team should determine what types of help materials users want.

information for the software. Support demands advance preparation. Feedback allows the customer to suggest changes that have business value and provide the developer with input for the next iterative software engineering cycle.

PROBLEMS AND POINTS TO PONDER

6.1. Because a focus on quality demands resources and time, is it possible to be agile and still maintain a quality focus?

6.2. Of the eight core principles that guide process (discussed in Section 6.1.1), which do you believe is most important?

6.3. Describe the concept of *separation of concerns* in your own words.

6.4. Why is it necessary to "move on"?

6.5. Do some research on "negotiation" for the communication activity, and prepare a set of guidelines that focus solely on negotiation.

6.6. Why are models important in software engineering work? Are they always necessary? Are there qualifiers to your answer about necessity?

6.7. What is a successful test?

6.8. Why is feedback important to the software team?

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

7 UNDERSTANDING REQUIREMENTS

Understanding problem requirements is among the most difficult tasks facing a software engineer. When you first think about it, developing a clear understanding of the requirements doesn't seem that hard. After all, doesn't the customer know what is required? Shouldn't the end users have a good understanding of the features and functions that they need? Surprisingly, in many instances, the answer to these questions is no. And even if customers and end users can state their needs explicitly, those needs will change throughout the project.

KEY CONCEPTS

analysis model 118
analysis patterns 122
collaboration 108
elaboration 104
elicitation 104
inception 104
negotiation 105
requirements engineering 103
requirements gathering 110
requirements management 106
requirements monitoring 123
specification 105
stakeholders 107
use cases 113
validating requirements 123
validation 105
viewpoints 107
work products 114



QUICK LOOK

What is it? Before you begin any technical work, it's a good idea to create a set of requirements for the engineering tasks. By establishing a set of requirements, you'll gain an understanding of what the business impact of the software will be, what the customer wants, and how end users will interact with the software.

Who does it? Software engineers and other project stakeholders (managers, customers, and end users) all participate in requirements engineering.

Why is it important? To understand what the customer wants before you begin to design and build a computer-based system. Building an elegant computer program that solves the wrong problem helps no one.

What are the steps? Requirements engineering begins with inception (a task that defines the scope and nature of the problem to be solved). It moves onward to elicitation (a task that helps stakeholders define what is required), and then elaboration (where basic requirements are refined and modified). As stakeholders define the problem, negotiation occurs (what are the priorities, what is essential, when is it required?). Finally, the problem is specified in some manner and then reviewed or validated to ensure that your understanding of the problem and the stakeholders' understanding of the problem coincide.

What is the work product? Requirements engineering provides all parties with a written understanding of the problem. The work products may include: usage scenarios, function and feature lists, and requirements models.

How do I ensure that I've done it right? Requirements engineering work products are reviewed with stakeholders to ensure that everyone is on the same page. A word of warning: Even after all parties agree, things will change, and they will continue to change throughout the project.

In the forward to a book by Ralph Young [You01] on effective requirements practices, one of us [RSP] wrote:

> It's your worst nightmare. A customer walks into your office, sits down, looks you straight in the eye, and says, "I know you think you understand what I said, but what you don't understand is what I said is not what I meant." Invariably, this happens late in the project, after deadline commitments have been made, reputations are on the line, and serious money is at stake.
>
> All of us who have worked in the systems and software business for more than a few years have lived this nightmare, and yet, few of us have learned to make it go away. We struggle when we try to elicit requirements from our customers. We have trouble understanding the information that we do acquire. We often record requirements in a disorganized manner, and we spend far too little time verifying what we do record. We allow change to control us, rather than establishing mechanisms to control change. In short, we fail to establish a solid foundation for the system or software. Each of these problems is challenging. When they are combined, the outlook is daunting for even the most experienced managers and practitioners. But solutions do exist.

It's reasonable to argue that the techniques we'll discuss in this chapter are not a true "solution" to the challenges just noted. But they do provide a solid approach for addressing these challenges.

7.1 REQUIREMENTS ENGINEERING

Designing and building computer software is challenging, creative, and just plain fun. In fact, building software is so compelling that many software developers want to jump right in before they have a clear understanding of what is needed. They argue that things will become clear as they build, that project stakeholders will be able to understand need only after examining early iterations of the software, that things change so rapidly that any attempt to understand requirements in detail is a waste of time, that the bottom line is producing a working program, and that all else is secondary. What makes these arguments seductive is that they contain elements of truth. But each argument is flawed and can lead to a failed software project.

Requirements engineering is the term for the broad spectrum of tasks and techniques that lead to an understanding of requirements. From a software process perspective, requirements engineering is a major software engineering action that begins during the communication activity and continues into the modeling activity. Requirements engineering establishes a solid base for design and construction. Without it, the resulting software has a high probability of not meeting customer's needs. It must be adapted to the needs of the process, the project, the product, and the people doing the work. It is important to realize that each of these tasks is done iteratively as the project team and the stakeholders continue to share information about their respective concerns.

Requirements engineering builds a bridge to design and construction. But where does the bridge originate? One could argue that it begins with the project stakeholders (e.g., managers, customers, and end users), where business needs are defined, user scenarios are described, functions and features are delineated, and project constraints are identified. Others might suggest that it begins with a broader system definition,

where software is but one component of the larger system domain. But regardless of the starting point, the journey across the bridge takes you high above the project, allowing you to examine the context of the software work to be performed; the specific needs that design and construction must address; the priorities that guide the order in which work is to be completed; and the information, functions, and behaviors that will have a profound impact on the resulting design. Requirements engineering encompasses seven tasks with sometimes muddy boundaries: *inception, elicitation, elaboration, negotiation, specification, validation,* and *management.* It is important to note that some of these tasks occur in parallel and all are adapted to the needs of the project. Expect to do a bit of design during requirements work and a bit of requirements work during design.

7.1.1 Inception

How does a software project get started? In general, most projects begin with an identified business need or when a potential new market or service is discovered. At project inception, you establish a basic understanding of the problem, the people who want a solution, and the nature of the solution that is desired. Communication between all stakeholders and the software team needs to be established during this task to begin an effective collaboration.

7.1.2 Elicitation

It certainly seems simple enough—ask the customer, the users, and others what the objectives for the system or product are, what is to be accomplished, how the system or product fits into the needs of the business, and finally, how the system or product is to be used on a day-to-day basis. But it isn't simple—it's very hard.

An important part of elicitation is to understand the business goals [Cle10]. A *goal* is a long-term aim that a system or product must achieve. Goals may deal with either functional or nonfunctional (e.g., reliability, security, usability) concerns [Lam09].

Goals are often a good way to explain requirements to stakeholders and, once established, can be used to manage conflicts among stakeholders. Goals should be specified precisely and serve as the basis for requirements elaboration, verification and validation, conflict management, negotiation, explanation, and evolution.

Your job is to engage stakeholders and to encourage them to share their goals honestly. Once the goals are captured, you establish a prioritization mechanism and create a design rationale for a potential architecture (that meets stakeholder goals).

Agility is an important aspect of requirements engineering. The intent of elicitation is to transfer ideas from stakeholders to the software team smoothly and without delay. It is highly likely that new requirements will continue to emerge as iterative product development occurs.

7.1.3 Elaboration

The elaboration task focuses on developing a refined requirements model that identifies various aspects of software function, behavior, and information (Chapter 8). Elaboration is driven by the creation and refinement of user scenarios obtained during elicitation. These scenarios describe how the end users (and other actors) will interact with the system. Each user scenario is parsed to extract *analysis classes*—business domain

entities that are visible to the end user. The attributes of each analysis class are defined, and the services that are required by each class are identified. The relationships and collaboration between classes are identified. Elaboration is a good thing, but you need to know when to stop. The key is to describe the problem in a way that establishes a firm base for design and then move on. Do not obsess over unnecessary details.

7.1.4 Negotiation

It isn't unusual for customers and users to ask for more than can be achieved, given limited business resources. It's also relatively common for different customers or users to propose conflicting requirements, arguing that their version is "essential for our special needs."

These conflicts need to be reconciled through the process of negotiation. Customers, users, and other stakeholders are asked to rank requirements and then discuss conflicts in priority. There should be no winner and no loser in an effective negotiation. Both sides win, because a "deal" that both can live with is solidified. You should use an iterative approach that prioritizes requirements, assesses their cost and risk, and addresses internal conflicts. In this way, requirements are eliminated, combined, and/or modified so that each party achieves some measure of satisfaction.

7.1.5 Specification

In the context of computer-based systems (and software), the term *specification* means different things to different people. A specification can be a written document, a set of graphical models, a formal mathematical model, a collection of usage scenarios, a prototype, or any combination of these.

Some suggest that a "standard template" [Som97] should be developed and used for a specification, arguing that this leads to requirements that are presented in a consistent and therefore more understandable manner. However, it is sometimes necessary to remain flexible when a specification is to be developed. The formality and format of a specification varies with the size and the complexity of the software to be built. For large systems, a written document, combining natural language descriptions and graphical models, may be the best approach. A template for a formal software requirements specification document can be downloaded from: https://web.cs.dal.ca/~hawkey/3130/srs_template-ieee.doc. However, usage scenarios may be all that are required for smaller products or systems that reside within well-understood technical environments.

7.1.6 Validation

The work products produced during requirements engineering are assessed for quality during a validation step. A key concern during requirements validation is consistency. Use the analysis model to ensure that requirements have been consistently stated. Requirements validation examines the specification to ensure that all software requirements have been stated unambiguously; that inconsistencies, omissions, and errors have been detected and corrected; and that the work products conform to the standards established for the process, the project, and the product.

The primary requirements validation mechanism is the *technical review* (Chapter 16). The review team that validates requirements includes software engineers, customers, users, and other stakeholders who examine the specification looking for errors in